

Relatório 7 - Prático: Se aprofundando no Blender Python (II)

Ana Clara Fortunato de Souza

Descrição da atividade

Introdução

O vídeo Python Crash Course for Blender!, apresentado por Curtis Holt, oferece uma introdução prática à criação de scripts dentro do Blender. O conteúdo parte da configuração do ambiente de trabalho e avança até a manipulação de objetos, aplicação de modificadores, suavização de malhas, criação de texturas e materiais por nós. Ao final, demonstra como organizar o código em um operador reutilizável, aproximando um script simples da estrutura de um add-on.

O uso de Python amplia as possibilidades do Blender porque converte ações manuais em procedimentos programáveis. Em vez de repetir alterações pela interface, é possível definir regras para criar, posicionar, rotacionar e modificar vários elementos automaticamente. Essa característica é particularmente importante na geração de dados sintéticos, em que uma mesma cena precisa produzir muitas imagens com variações controladas de geometria, aparência, iluminação e câmera.

Preparação do ambiente de programação

O espaço de trabalho Scripting reúne o editor de texto, o console Python, a visualização 3D e painéis de propriedades. Essa organização permite escrever o código e observar imediatamente seus efeitos na cena. O vídeo também apresenta os Python Tooltips, que exibem o caminho de uma propriedade ao posicionar o cursor sobre um controle da interface, e a janela Info, útil para identificar comandos executados pelo Blender.

Esses recursos facilitam o aprendizado da API, pois ajudam a relacionar uma ação visual ao comando correspondente. No editor de texto, o módulo bpy deve ser importado para permitir o acesso aos dados e operações do Blender; no console interno, ele normalmente já está disponível.

Estrutura básica da API Python do Blender

A API do Blender é organizada em conjuntos com funções diferentes. Compreender essa separação torna os scripts mais claros e reduz erros relacionados ao objeto ativo, à seleção ou ao modo de edição.

Elemento	Função principal
bpy.data	Acessa blocos de dados do arquivo, como objetos, malhas, materiais, texturas, imagens e cenas.
bpy.context	Representa o estado atual do Blender, incluindo cena, modo, seleção e objeto ativo.
bpy.ops	Disponibiliza operadores semelhantes às ações executadas pela interface.
bpy.types	Fornecer os tipos usados para criar operadores, painéis e outras estruturas de add-ons.

Tabela 1 - Principais componentes do módulo bpy. Fonte: elaboração própria (2026).

Criação, acesso e transformação de objetos

A criação de um cubo pode ser feita com `bpy.ops.mesh.primitive_cube_add()`. Como o novo objeto passa a ser o objeto ativo, ele pode ser armazenado em uma variável por meio de `bpy.context.active_object`. A partir dessa referência, suas propriedades podem ser alteradas diretamente, incluindo nome, localização, escala e rotação.

```
import bpy
from math import radians

bpy.ops.mesh.primitive_cube_add(size=2, location=(0, 0, 0))
cubo = bpy.context.active_object
cubo.name = 'CuboAzul'
cubo.location = (1.0, 0.0, 0.5)
cubo.rotation_euler[2] = radians(45)
```

Um ponto importante é que as rotações são representadas internamente em radianos. Por isso, a função `radians()`, do módulo `math`, facilita a conversão de valores em graus. Essa manipulação programática permite variar automaticamente a posição e a orientação dos objetos, o que pode ser aproveitado para gerar cenas com diferentes pontos de vista e composições.

Modificadores, suavização e texturas

Os modificadores alteram a geometria de forma não destrutiva. No vídeo, o objeto recebe `Subdivision Surface` para aumentar a quantidade de polígonos e `Displace` para deformar a superfície a partir de uma textura procedural. A suavização das faces reduz o aspecto facetado e melhora a aparência visual da malha.

```
subsurf = cubo.modifiers.new(name='Subdivisão', type='SUBSURF')
subsurf.levels = 2

bevel = cubo.modifiers.new(name='Bordas', type='BEVEL')
bevel.width = 0.08
bevel.segments = 3

for face in cubo.data.polygons:
    face.use_smooth = True
```

Esses recursos são relevantes para a criação procedural porque seus parâmetros podem ser alterados por código. Assim, um único modelo-base pode originar diversas variações geométricas, desde que os limites definidos preservem a coerência do objeto representado.

Atividade prática: criação de um cubo azul

Na atividade prática, foi criado um cubo e aplicado um material azul utilizando o sistema de nós do Blender. O material foi configurado com o Principled BSDF, alterando a entrada Base Color e o valor de rugosidade. Essa etapa demonstrou que a aparência do objeto também pode ser controlada por script, sem a necessidade de configurar manualmente cada propriedade no painel de materiais.

```
mat = bpy.data.materials.new(name='MeuAzul')
mat.use_nodes = True

principled = mat.node_tree.nodes.get('Principled BSDF')
principled.inputs['Base Color'].default_value = (0.05, 0.25, 0.8, 1.0)
principled.inputs['Roughness'].default_value = 0.4

cubo.data.materials.append(mat)
```

Durante o teste, o cubo continuou aparentemente cinza no modo de visualização Solid, embora o material tivesse sido criado corretamente. Isso ocorre porque esse modo não exibe necessariamente a cor real do material. Ao selecionar Material Preview ou Rendered, a cor definida no Principled BSDF passa a ser apresentada. O comportamento reforça a importância de distinguir a configuração do material da forma como a viewport exibe a cena.

Materiais baseados em nós

Ao ativar `use_nodes`, o material passa a utilizar uma árvore de nós. O nó Principled BSDF concentra propriedades físicas como cor base, rugosidade e metalicidade, enquanto o Material Output recebe o resultado final do shader. Também é possível criar nós de emissão, texturas e operações matemáticas, além de estabelecer suas conexões pelo próprio script.

Para dados sintéticos, esse controle permite variar cores, níveis de brilho, reflexo, textura e transparência dentro de intervalos definidos. A diversidade visual resultante pode reduzir a dependência de um único padrão de aparência e contribuir para modelos de visão computacional mais robustos.

Operadores e reutilização do código

A parte final do vídeo transforma a sequência de comandos em uma classe derivada de `bpy.types.Operator`. O método `execute` reúne as ações que serão realizadas; `bl_idname` identifica o operador; `bl_label` define o nome apresentado ao usuário; e as funções `register()` e `unregister()` controlam seu registro no Blender. Propriedades como `FloatProperty` tornam os valores ajustáveis pela interface.

Essa organização é vantajosa quando um mesmo procedimento será executado várias vezes ou utilizado por outras pessoas. Em lugar de editar o código a cada teste, o usuário pode alterar apenas os parâmetros expostos pelo operador, tornando a automação mais acessível e reduzindo erros de configuração.

Relação com a geração de dados sintéticos

Os conceitos apresentados constituem a base de um pipeline automatizado. Um script pode preparar a cena, sortear valores dentro de limites definidos, atualizar objetos e materiais, renderizar a imagem e salvar os parâmetros empregados. Quando esse ciclo é repetido, o Blender deixa de ser apenas uma ferramenta de modelagem e passa a atuar como uma plataforma programável para produção de datasets.



Figura 1 - Fluxo simplificado de automação de cenas no Blender.

Fonte: elaboração própria (2026).

Entre as possibilidades estão a alteração automática de posição e rotação dos objetos, a variação de materiais e texturas, a mudança de iluminação, o reposicionamento da câmera e a exportação de anotações. Para garantir reprodutibilidade, é recomendável registrar as sementes aleatórias, os valores usados, a versão do script e o nome de cada arquivo gerado.

Conhecimentos adquiridos

- configurar o ambiente Scripting e utilizar os Python Tooltips como apoio para descobrir propriedades da API;
- diferenciar o acesso a dados, ao contexto atual e aos operadores por meio de `bpy.data`, `bpy.context` e `bpy.ops`;
- criar, localizar, mover, rotacionar e renomear objetos por código;
- adicionar e configurar modificadores, além de aplicar suavização à malha;
- criar materiais baseados em nós e controlar cor e rugosidade pelo Principled BSDF;
- compreender como operadores e propriedades personalizadas tornam o código reutilizável;
- relacionar a automação de cenas aos requisitos de escala e repetibilidade dos dados sintéticos.

Dificuldades

A principal dificuldade foi perceber que a criação correta do material não garante que sua cor seja mostrada no modo Solid. Foi necessário alterar a viewport para Material Preview ou Rendered para visualizar o cubo azul. Também exigiram atenção a identificação do objeto ativo e o uso de radianos nas rotações. Os testes em pequenos trechos e a consulta aos Python Tooltips ajudaram a confirmar os caminhos das propriedades e corrigir essas diferenças.

Conclusões

O vídeo apresentou uma base prática para automatizar tarefas no Blender com Python. A criação e manipulação de objetos, o uso de modificadores, a suavização de malhas e a configuração de materiais demonstram que ações realizadas manualmente podem ser convertidas em procedimentos repetíveis e parametrizados.

A prática do cubo azul permitiu compreender tanto a criação do material por nós quanto a influência do modo de visualização da viewport. Além disso, a introdução aos operadores mostrou como um script pode evoluir para uma ferramenta reutilizável. Esses conhecimentos são diretamente aplicáveis à geração de dados sintéticos, pois permitem criar variações de cena em escala, com maior controle, consistência e reprodutibilidade.

Referências

HOLT, Curtis. **Python Crash Course for Blender!** YouTube, 3 jul. 2020. Disponível em: <https://youtu.be/XqX5wh4YeRw>. Acesso em: 3 ago. 2026.

BLENDER FOUNDATION. **Blender Python API Documentation**. Disponível em: <https://docs.blender.org/api/current/>. Acesso em: 3 ago. 2026.

BLENDER FOUNDATION. **Introduction to Nodes. Blender Manual**. Disponível em: <https://docs.blender.org/manual/en/latest/interface/controls/nodes/introduction.html>. Acesso em: 3 ago. 2026.

BLENDERNATION. **Python Crash Course for Blender!** 3 jul. 2020. Disponível em: <https://www.blendernation.com/2020/07/03/python-crash-course-for-blender/>. Acesso em: 3 ago. 2026.